

**MID 15 September 2026**

## **Theory Part(6Marks)**

### **1. Normalization — Theory**

**Normalization** converts raw text into a consistent form before NLP processing.

For example:

HELLOOO!!!      আজকে ১০০%

may become:

helloooo!      আজকে 100%

The code performs:

- Unicode normalization
- Lowercasing
- Whitespace normalization
- Bangla digit → English digit conversion
- Repeated punctuation reduction
- URL replacement
- Email replacement

#### **Example output**

```
hello! ami nlp shikhtesi...
আজকে 100% excited!
check this: <URL>
email: <EMAIL>
i can't believe it's sooo good!
#nlp #machinelearning
```

---

### **2. Sentence Tokenization — Theory**

**Sentence tokenization** divides a paragraph into individual sentences.

The code looks for:

.  
!  
?

#### **Example**

Input:

I love NLP! NLP is interesting. Do you like it?

Output:

```
1 : I love NLP!
2 : NLP is interesting.
3 : Do you like it?
```

---

## 3. Word Tokenization — Theory

**Word tokenization** divides a sentence into individual tokens.

For example:

I love NLP!

becomes:

```
["I", "love", "NLP", "!"]
```

Notice that punctuation such as ! is also treated as a token.

Your regex:

```
r'\w+|[\^\w\s]'
```

basically means:

- `\w+` → words/numbers
  - `[\^\w\s]` → characters that aren't words or whitespace
- 

## 4. Morphology — Theory

**Morphology** is the study of the internal structure of words.

A word can be divided into:

Prefix + Root + Suffix

For example:

un + happy

Here:

- un → prefix

- happy → root
- unhappy → complete word

Another example:

play + ing

Here:

- play → root
- ing → suffix
- playing → complete word

## Morphology output

Your program will produce approximately:

|                         |              |              |             |       |
|-------------------------|--------------|--------------|-------------|-------|
| unhappy                 | → Prefix: un | Root: happy  | Suffix: -   | Type: |
| Derivation              |              |              |             |       |
| unhelpful               | → Prefix: un | Root: help   | Suffix: ful | Type: |
| Derivation              |              |              |             |       |
| playing                 | → Prefix: -  | Root: play   | Suffix: ing | Type: |
| Inflection              |              |              |             |       |
| played                  | → Prefix: -  | Root: play   | Suffix: ed  | Type: |
| Inflection              |              |              |             |       |
| players                 | → Prefix: -  | Root: play   | Suffix: ers | Type: |
| Derivation + Inflection |              |              |             |       |
| cats                    | → Prefix: -  | Root: cat    | Suffix: s   | Type: |
| Inflection              |              |              |             |       |
| beautiful               | → Prefix: -  | Root: beauty | Suffix: ful | Type: |
| Derivation              |              |              |             |       |

## Important theory: Inflection vs Derivation

**Inflection** changes grammatical information but usually doesn't create a new dictionary word.

cat → cats  
 play → played  
 play → playing

**Derivation** creates a new word or changes its grammatical category/meaning.

happy → unhappy  
 help → helpful  
 beauty → beautiful

---

# 5. Stemming — Theory

**Stemming** removes prefixes/suffixes using simple rules to obtain a stem.

It does **not necessarily produce a valid English word**.

For example:

```
playing → play
played  → play
cars    → car
```

Your manual stemmer checks suffixes:

```
for suffix in suffixes:

    if word.endswith(suffix) and \
        len(word) > len(suffix) + 2:

        return word[:-len(suffix)]
```

Suppose:

```
word = "playing"
suffix = "ing"
```

Then:

```
word.endswith("ing")
```

is:

```
True
```

and:

```
word[:-3]
```

removes the final 3 characters:

```
playing
```

→

```
play
```

## Expected output

```
playing → play
played  → play
plays   → play
beautifully → beautiful
cars    → car
running → runn
```

Notice something important:

```
running → runn
```

This is a **limitation of your simple stemmer**.

A real stemming algorithm may have additional rules to handle this better.

---

## 6. Lemmatization — Theory

**Lemmatization** converts a word to its proper **base/dictionary form**, called the **lemma**.

Unlike stemming, lemmatization tries to produce a meaningful word.

Examples:

```
running → run
ran      → run
children → child
mice     → mouse
better   → good
was      → be
```

Your program uses a manually created dictionary:

```
lemma_dictionary = {
    "running": "run",
    "ran": "run",
    "children": "child",
    "better": "good"
}
```

### Expected output

```
running → run
ran → run
cars → car
children → child
better → good
was → be
```

---

## 7. Stemming vs Lemmatization

This is a very important exam question.

| Stemming                             | Lemmatization                          |
|--------------------------------------|----------------------------------------|
| Uses simple rules                    | Uses vocabulary/linguistic knowledge   |
| Removes suffixes/prefixes            | Finds dictionary/base form             |
| Faster                               | Usually more computationally expensive |
| May produce invalid words            | Usually produces valid words           |
| running → runn in our simple example | running → run                          |

### Stemming

better → better

### Lemmatization

better → good

## Easy way to remember

### Stemming = Cut

playing → play

### Lemmatization = Understand

better → good

---

## 8. Sentiment Analysis — Theory

Sentiment analysis determines whether text expresses:

- Positive sentiment
- Negative sentiment
- Neutral sentiment

Your program uses a **dictionary-based approach**.

For example:

অসাধারণ → Positive

চমৎকার → Positive

খারাপ → Negative

বাজে → Negative

For:

সিনেমাটি অসাধারণ এবং চমৎকার  
কিন্তু কিছুটা বাজে ও খারাপ

The program finds:

Positive words:

['অসাধারণ', 'চমৎকার']

Negative words:

['বাজে', 'খারাপ']

Therefore:

Positive count = 2

Negative count = 2

Output:

## 9. PyTorch Representation — Theory

Machine-learning models cannot directly understand:

```
["I", "love", "NLP"]
```

They need numerical representations.

Your function creates a vocabulary:

```
I → 1  
love → 2  
NLP → 3
```

Then:

```
["I", "love", "NLP"]
```

becomes:

```
[1, 2, 3]
```

and PyTorch converts it into:

```
tensor([1, 2, 3])
```

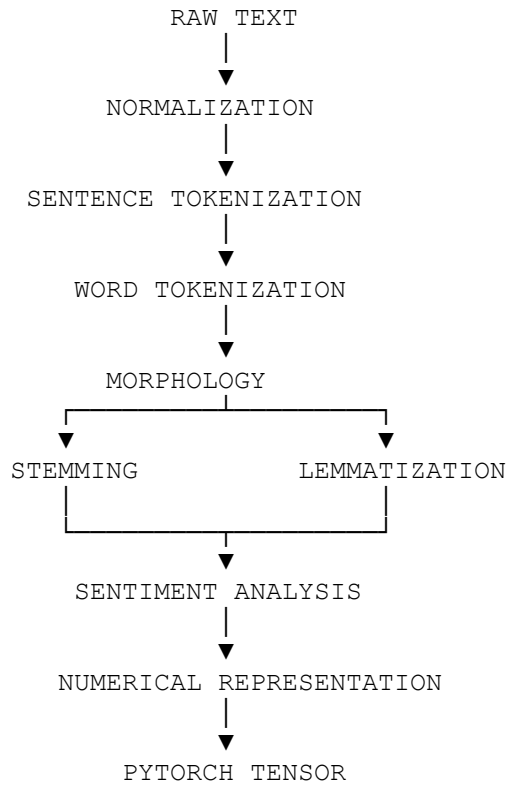
with:

```
dtype=torch.long
```

---

# Overall pipeline for your exam

You can memorize this diagram:



## CODE PART(14 Marks)

```
import re
import unicodedata
import torch

#
=====

# 1. NORMALIZATION
#
=====

def normalize_text(text):

    # Unicode normalization
    text = unicodedata.normalize("NFC", text)

    # Convert English uppercase to lowercase
    text = text.lower()

    # Normalize whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Bangla digits → English digits
    bangla_digits = "০১২৩৪৫৬৭৮৯"
    english_digits = "0123456789"
```

```
translation_table = str.maketrans(  
    bangla_digits,  
    english_digits  
)
```

```
text = text.translate(translation_table)
```

```
# Normalize repeated punctuation
```

```
text = re.sub(r'!\{2,}', '!', text)
```

```
text = re.sub(r'\?\{2,}', '?', text)
```

```
text = re.sub(r'\.\{3,}', '...', text)
```

```
# Replace URLs
```

```
text = re.sub(  
    r'https?://\S+|www\.\S+',  
    '<URL>',  
    text  
)
```

```
# Replace email addresses
```

```
text = re.sub(  
    r'\S+@\S+\.\S+',  
    '<EMAIL>',  
    text  
)
```

```
return text
```

```
#
```

```
=====
```

```
# 2. SENTENCE TOKENIZATION
```

```
#
```

```
=====
```

```
def sentence_tokenize(text):
```

```
    # Split after ., ! or ?
```

```
    sentences = re.split(
```

```
        r'(?<=[.!?])\s+',
```

```
        text
```

```
    )
```

```
    # Remove empty strings
```

```
    sentences = [
```

```
        sentence.strip()
```

```
        for sentence in sentences
```

```
        if sentence.strip()
```

```
    ]
```

```
    return sentences
```

```
#
=====

# 3. WORD TOKENIZATION

#
=====
```

```
def word_tokenize(text):

    # Words, numbers, symbols and punctuation
    tokens = re.findall(
        r'\w+|[\^\w\s]',
        text,
        flags=re.UNICODE
    )

    return tokens
```

```
#
=====

# 4. MORPHOLOGY ANALYSIS

#
=====
```

```
def morphology_analysis(word):

    """

    Simple rule-based morphology analyzer.
```

Identifies:

- Prefix
- Root
- Suffix
- Type

"""

```
word_lower = word.lower()
```

```
# -----
```

```
# English examples
```

```
# -----
```

```
morphology_rules = {
```

```
    "unhappy": {
        "prefix": "un",
        "root": "happy",
        "suffix": "",
        "type": "Derivation"
    },
```

```
    "unhelpful": {
        "prefix": "un",
        "root": "help",
        "suffix": "ful",
        "type": "Derivation"
```

},

```
"playing": {  
  "prefix": "",  
  "root": "play",  
  "suffix": "ing",  
  "type": "Inflection"  
},
```

```
"played": {  
  "prefix": "",  
  "root": "play",  
  "suffix": "ed",  
  "type": "Inflection"  
},
```

```
"players": {  
  "prefix": "",  
  "root": "play",  
  "suffix": "ers",  
  "type": "Derivation + Inflection"  
},
```

```
"cats": {  
  "prefix": "",  
  "root": "cat",  
  "suffix": "s",
```

```

        "type": "Inflection"
    },

    "beautiful": {
        "prefix": "",
        "root": "beauty",
        "suffix": "ful",
        "type": "Derivation"
    }
}

if word_lower in morphology_rules:
    return morphology_rules[word_lower]

# If no rule is found
return {
    "prefix": "",
    "root": word_lower,
    "suffix": "",
    "type": "Unknown"
}

```

```

#
=====

# 5. SIMPLE MANUAL STEMMING

#
=====

```

```

def manual_stem(word):

    # Very simple English suffix rules
    suffixes = [
        "ingly",
        "edly",
        "ing",
        "ed",
        "ly",
        "es",
        "s"
    ]

    for suffix in suffixes:

        if word.endswith(suffix) and \
            len(word) > len(suffix) + 2:

            return word[:-len(suffix)]

    return word

#
=====

# 6. MANUAL LEMMATIZATION

```

#

=====

lemma\_dictionary = {

    "am": "be",

    "is": "be",

    "are": "be",

    "was": "be",

    "were": "be",

    "running": "run",

    "ran": "run",

    "runs": "run",

    "eating": "eat",

    "ate": "eat",

    "eats": "eat",

    "children": "child",

    "mice": "mouse",

    "better": "good",

    "best": "good",

    "cars": "car",

    "books": "book"

}

```
def manual_lemmatize(word):
```

```
    word_lower = word.lower()
```

```
    if word_lower in lemma_dictionary:
```

```
        return lemma_dictionary[word_lower]
```

```
    return word_lower
```

```
#
```

```
=====
```

```
# 7. SENTIMENT ANALYSIS
```

```
#
```

```
=====
```

```
def simple_sentiment(text):
```

```
    positive_words = [
```

```
        "অসাধারণ",
```

```
        "চমৎকার",
```

```
        "ভালো",
```

```
        "সুন্দর",
```

```
        "ভাল",
```

```
        "great",
```

```
"excellent",  
"good",  
"amazing",  
"beautiful"  
]
```

```
negative_words = [  
    "বাজে",  
    "বিরক্তিকর",  
    "ভয়ংকর",  
    "খারাপ",  
    "bad",  
    "terrible",  
    "horrible",  
    "boring"  
]
```

```
tokens = word_tokenize(text)
```

```
positive_found = []  
negative_found = []
```

```
for token in tokens:
```

```
    if token in positive_words:  
        positive_found.append(token)
```

```

        if token in negative_words:
            negative_found.append(token)

positive_count = len(positive_found)
negative_count = len(negative_found)

if positive_count > negative_count:
    sentiment = "Positive"

elif negative_count > positive_count:
    sentiment = "Negative"

else:
    sentiment = "Neutral or Unknown"

return {
    "positive_words": positive_found,
    "negative_words": negative_found,
    "positive_count": positive_count,
    "negative_count": negative_count,
    "sentiment": sentiment
}

```

#

=====

# 8. PYTORCH REPRESENTATION

```
#
```

```
=====
```

```
def tokens_to_tensor(tokens):
```

```
    # Create vocabulary manually
```

```
    vocabulary = {}
```

```
    for token in tokens:
```

```
        if token not in vocabulary:
```

```
            vocabulary[token] = len(vocabulary) + 1
```

```
    # Convert tokens → integer IDs
```

```
    token_ids = [
```

```
        vocabulary[token]
```

```
        for token in tokens
```

```
    ]
```

```
    # Convert to PyTorch tensor
```

```
    tensor = torch.tensor(
```

```
        token_ids,
```

```
        dtype=torch.long
```

```
    )
```

```
    return vocabulary, tensor
```

#

=====

# COMPLETE PIPELINE

#

=====

text = ""

HELLOOO!!! 😊😊

Ami NLP shikhtesi...

আজকে ১০০% excited!!!

Check this: <HTTPS://Example.COM/NLP?id=123> 😊

Email: STUDENT@GMAIL.COM

I can't believe it's sooo good!!!

#NLP #MachineLearning

""

#

=====

# STEP 1: ORIGINAL TEXT

#

=====

print("=" \* 60)

print("STEP 1: ORIGINAL TEXT")

print("=" \* 60)

print(text)

```
#
=====

# STEP 2: NORMALIZATION
#
=====

normalized = normalize_text(text)

print("\n" + "=" * 60)
print("STEP 2: NORMALIZED TEXT")
print("=" * 60)

print(normalized)

#
=====

# STEP 3: SENTENCE TOKENIZATION
#
=====

sentences = sentence_tokenize(normalized)

print("\n" + "=" * 60)
print("STEP 3: SENTENCE TOKENIZATION")
print("=" * 60)
```

```
for i, sentence in enumerate(sentences, 1):
```

```
    print(i, ":", sentence)
```

```
#
```

```
=====
```

```
# STEP 4: WORD TOKENIZATION
```

```
#
```

```
=====
```

```
tokens = word_tokenize(normalized)
```

```
print("\n" + "=" * 60)
```

```
print("STEP 4: WORD TOKENIZATION")
```

```
print("=" * 60)
```

```
print(tokens)
```

```
#
```

```
=====
```

```
# STEP 5: MORPHOLOGY
```

```
#
```

```
=====
```

```
print("\n" + "=" * 60)
```

```
print("STEP 5: MORPHOLOGY ANALYSIS")
```

```
print("=" * 60)
```

```
morphology_words = [  
    "unhappy",  
    "unhelpful",  
    "playing",  
    "played",  
    "players",  
    "cats",  
    "beautiful"  
]
```

```
for word in morphology_words:
```

```
    result = morphology_analysis(word)
```

```
    print(  
        f"{word:12} → "  
        f"Prefix: {result['prefix'] or '-':5} | "  
        f"Root: {result['root']:8} | "  
        f"Suffix: {result['suffix'] or '-':5} | "  
        f"Type: {result['type']}"  
    )
```

```
#
```

```
=====
```

```
# STEP 6: STEMMING
```

```
#
```

```
=====
```

```
print("\n" + "=" * 60)
```

```
print("STEP 6: STEMMING")
```

```
print("=" * 60)
```

```
stemming_words = [
```

```
    "playing",
```

```
    "played",
```

```
    "plays",
```

```
    "beautifully",
```

```
    "cars",
```

```
    "running"
```

```
]
```

```
for word in stemming_words:
```

```
    print(
```

```
        word,
```

```
        "→",
```

```
        manual_stem(word)
```

```
    )
```

```
#
```

```
=====
```

```
# STEP 7: LEMMATIZATION
```

```
#
```

```
=====
```

```
print("\n" + "=" * 60)
```

```
print("STEP 7: LEMMATIZATION")
```

```
print("=" * 60)
```

```
lemmatization_words = [
```

```
    "running",
```

```
    "ran",
```

```
    "cars",
```

```
    "children",
```

```
    "better",
```

```
    "was"
```

```
]
```

```
for word in lemmatization_words:
```

```
    print(
```

```
        word,
```

```
        "→",
```

```
        manual_lemmatize(word)
```

```
    )
```

```
#
```

```
=====
```

```
# STEP 8: SENTIMENT ANALYSIS
```

```

#
=====

print("\n" + "=" * 60)
print("STEP 8: SENTIMENT ANALYSIS")
print("=" * 60)

sentiment_text = (
    "সিনেমাটি অসাধারণ এবং চমৎকার "
    "কিন্তু কিছুটা বাজে ও খারাপ"
)

sentiment_result = simple_sentiment(
    sentiment_text
)

print(sentiment_result)

#
=====

# STEP 9: PYTORCH REPRESENTATION
#
=====

print("\n" + "=" * 60)
print("STEP 9: PYTORCH REPRESENTATION")
print("=" * 60)

```

```
vocab, tensor = tokens_to_tensor(tokens)
```

```
print("Vocabulary:")
```

```
print(vocab)
```

```
print("\nToken IDs:")
```

```
print(tensor)
```